Informe — Laboratorio 3

Funciones, módulos y programación orientada a objetos en Python

Aprendiz	Anthony Córdova Calderón
Ficha	3388087
Usuario de GitHub	skyyleerr
Repositorio	https://github.com/skyyleerr/python-laboratorios
Carpeta del laboratorio	laboratorio-03/ (proyecto en laboratorio-03/gestor_inventario/)
Informe dentro del repositorio	laboratorio-03/informe/Informe_Laboratorio_03.pdf

El repositorio es el mismo de los laboratorios 01 y 02: ambos se conservan sin cambios en **laboratorio-01/** y **laboratorio-02/**, de modo que se puede comparar el gestor procedural (`laboratorio-02/gestor_inventario.py`) con su transformación orientada a objetos.

1. Descripción del gestor de inventario orientado a objetos

Es una aplicación de consola que registra, lista, busca, actualiza y elimina productos, y genera dos informes: productos sin existencias y resumen del inventario (cantidad de productos, unidades, valor total y categorías utilizadas). Conserva el menú y las funcionalidades del Laboratorio 2, pero cada producto ahora es un objeto **Producto** y la colección es administrada por un objeto **Inventario**. La información permanece en memoria mientras el programa está abierto.

El proyecto está dividido en tres módulos con responsabilidades distintas:

Archivo	Responsabilidad	¿Usa input() / print()?
producto.py	Clase Producto: datos, validaciones desde el constructor, propiedades, existencias y valor.	No
inventario.py	Clase Inventario: colección propia de productos, operaciones y reportes.	No
main.py	Menú, lectura de datos con reintento, captura de ValueError y mensajes al usuario.	Sí (único módulo)
pruebas_independencia.py	Demuestra que dos Inventario mantienen colecciones separadas.	Solo print()
test_gestor.py / pruebas_obligatorias.py	13 pruebas unitarias y 18 pruebas del enunciado (con input() simulado).	—
gestor_depuracion.py	Seis errores provocados y corregidos, con la causa explicada.	Solo print()

2. Clases, atributos, métodos y responsabilidades

2.1 Clase Producto (producto.py)

Representa un artículo y **controla sus valores**. Las reglas se aplican en `__init__`, antes de guardar cualquier atributo, de modo que no puede existir un objeto Producto con datos inválidos. Si algo no cumple, se lanza `ValueError` y `main.py` muestra un mensaje.

Elemento	Detalle
Atributos internos	_codigo, _nombre, _categoria, _precio, _cantidad (el guion bajo indica uso interno por convención).
Constante	CATEGORIAS = ("Periféricos", "Componentes", "Accesorios"), declarada una sola vez.
Propiedades	codigo, nombre, categoria, precio, cantidad: solo consulta. Sin setter para código; asignar producto.codigo = "X" produce AttributeError.
Validaciones	Código y nombre no vacíos (ni solo espacios); categoría autorizada; precio finito y > 0; cantidad entera ≥ 0. El precio se guarda como float.
actualizar_existencias(nueva)	Devuelve False (sin modificar nada) si el valor no es un entero ≥ 0; si es válido, actualiza y devuelve True.
calcular_valor()	Devuelve precio × cantidad. No imprime: el resultado se reutiliza en el resumen.
__str__()	Formato de listado: P001 - Teclado - Periféricos - $85000.0 - 4 unidades.

2.2 Clase Inventario (inventario.py)

Conserva varios objetos Producto en `self._productos = []`, creado dentro de `__init__`, por lo que cada instancia tiene su propia lista.

Método	Qué hace y qué devuelve
registrar_producto(producto)	Rechaza (False) lo que no sea Producto o repita un código; si no, agrega y devuelve True.
listar_productos()	Devuelve una copia de la lista interna: modificar la copia no altera el inventario.
buscar_por_codigo(codigo)	Devuelve el Producto o None. Es una consulta: no cambia la colección.
actualizar_existencias(codigo, n)	Busca el producto y delega la regla en Producto.actualizar_existencias; devuelve bool.
eliminar_producto(codigo)	Quita el producto si existe (True); si no, devuelve False sin error.
productos_sin_stock()	Comprensión de lista: productos con cantidad == 0.
resumen()	Diccionario con productos_registrados, unidades_disponibles, valor_total y categorias (comprensión de conjunto).
Ampliación	buscar_por_categoria, listar_ordenados_por_nombre, producto_mayor_valor, mapa_codigos_nombres (comprensión de diccionario).

2.3 main.py

Contiene las funciones de interacción: `mostrar_menu`, `leer_texto`, `leer_precio`, `leer_cantidad` (reintentan hasta recibir un dato válido sin detener el programa), `convertir_precio` (acepta *85000*, *85.000* y *85.000,50*) y una función por opción del menú. Decide qué mensaje se muestra después de cada operación según el booleano o el objeto que reciben de Inventario.

3. Estado del proyecto y resultados de las pruebas

Estado: completo. Las siete opciones del menú y la opción de salida funcionan; se cumplen las reglas de Producto e Inventario, y se incluye el reto de ampliación y el de depuración. Todas las pruebas se ejecutaron en Python 3.12 desde la carpeta del proyecto.

Comando	Resultado
python -m unittest -v	13 pruebas, OK

Comando	Resultado
python pruebas_obligatorias.py	18 pruebas, OK
python pruebas_independencia.py	Primero: 1 / Segundo: 0
python gestor_depuracion.py	Seis errores reproducidos y corregidos
python main.py (flujo manual con los tres productos del enunciado)	Resultados numéricos correctos (ver 3.2)

3.1 Pruebas obligatorias del enunciado

Caso	Resultado obtenido	Estado
Inventario vacío	«El inventario está vacío.» y resumen con 0 productos y 0 unidades	OK
Registrar P001	«Producto registrado.»	OK
Registrar P001 de nuevo	«No se pudo registrar: el código ya existe.» No se agrega nada	OK
Código o nombre vacío / solo espacios	«Este campo no puede estar vacío.» y se vuelve a pedir; no se guardan registros parciales	OK
Categoría no permitida	Rechazada con mensaje; inventario sigue vacío	OK
Precio cero o negativo	«El precio debe ser mayor que cero.» y nueva oportunidad	OK
Cantidad negativa	«La cantidad no puede ser negativa.» y nueva oportunidad	OK
Cantidad cero	Producto válido; aparece en «sin stock»	OK
Buscar P001 / P999	Muestra el producto / «Producto no registrado.»	OK
Actualizar a 10	Cambia solo la cantidad (4 → 10); precio y nombre intactos	OK
Actualizar a -3	Se rechaza y se conserva el dato anterior	OK
Eliminar existente / inexistente	«Producto eliminado.» / «Producto no registrado; no se eliminó nada.»	OK
Texto en campo numérico	«Debes escribir un número válido / entero.» y nueva oportunidad	OK
Resumen	Totales coherentes con los datos actuales	OK
Salir y opción no válida	«Programa finalizado.» / «Opción no válida.»	OK

3.2 Comprobación numérica

Con P001 (85000 × 4), P002 (45000 × 0) y P003 (700000 × 2), el programa muestra:

```
Productos registrados: 3
Unidades disponibles: 6
Valor del inventario: $1740000.0
Categorías utilizadas: Componentes, Periféricos      (Cantidad de categorías: 2)
Sin existencias: - Mouse

Tras actualizar P002 a 5 unidades:
Productos registrados: 3 | Unidades disponibles: 11 | Valor del inventario: $1965000.0
```

Coinciden con los valores esperados por el laboratorio: 340000 + 0 + 1400000 = 1740000, y 1740000 + 225000 = 1965000.

3.3 Prueba de independencia

```
primero = Inventario(); segundo = Inventario()
primero.registrar_producto(Producto("P001", "Teclado", "Periféricos", 85000, 4))
print("Primero:", len(primero.listar_productos()))   # 1
print("Segundo:", len(segundo.listar_productos()))   # 0
```

4. Seguimiento de commits y decisiones de diseño

Los commits siguen el orden de la transformación: primero las prácticas (funciones → módulos → clases → encapsulamiento), después las clases del proyecto integrador, el menú y por último las pruebas y la documentación. Los hashes corresponden a los commits reales del repositorio.

Commit	Mensaje	Justificación
f8b74c7	docs: README principal con los tres laboratorios	Deja visibles los tres laboratorios y cómo ejecutarlos.
4f26075	feat(lab03): practica 3.1 funciones, retorno, alcance y colecciones	Primero se extraen las tareas a funciones con retorno; sienta la base para separar cálculo y presentación.
2b89785	feat(lab03): practica 3.2 modulos, importaciones y cotizador modular	Se reparte el código en módulos y se protege la importación con if __name__ == "__main__".
01de359	feat(lab03): practica 3.3 clases, __init__, self y atributos de instancia	Se pasa de diccionarios a objetos y se comprueba por qué la lista debe crearse en __init__.
f90a164	feat(lab03): practica 3.4 encapsulamiento, propiedades y colaboracion de clases	Se introduce encapsulamiento y se prueba la colaboración Inventario–Producto.
cc87a02	feat(gestor): clase Producto con validaciones desde el constructor y propiedades	Validación en el constructor (no solo al actualizar) para que un producto nunca nazca inválido.
6570336	feat(gestor): clase Inventario con coleccion propia por instancia	self._productos en __init__ y duplicados controlados aquí, porque Inventario conoce la colección.
17fb89a	feat(gestor): main.py con menu y lectura de datos separados de las clases	Todo input()/print() queda fuera de las clases; main solo pide datos e informa.
c55e225	test(gestor): demostrar que dos Inventario mantienen colecciones independientes	Evidencia de que dos instancias no comparten datos.
f9efb87	test(gestor): pruebas unitarias de Producto e Inventario	Protegen las reglas de las clases ante futuros cambios.
f4a9de7	test(gestor): pruebas obligatorias del enunciado simulando input()	Reproducen la tabla de pruebas del enunciado simulando el teclado.
bce60d8	docs(gestor): reto de depuracion con seis errores explicados	Seis errores típicos reproducidos en el reto de depuración, cada uno con su causa.
d3ec41e	docs(lab03): READMEs con responsabilidades, ejecucion y pruebas	Documentan responsabilidades, ejecución y pruebas.

4.1 Decisiones de diseño, reorganización y correcciones

- **Validar en el constructor.** En la práctica 3.4 el constructor asignaba los atributos directamente y solo el setter validaba. En el proyecto, `Producto.__init__` valida todo antes de asignar y usa `ValueError`; `main.py` la captura para dar un mensaje comprensible.
- **Código de solo lectura.** Se ofrece propiedad sin setter para que el identificador no cambie después de registrarse.
- **Rechazar tipos incorrectos.** Se descartan `bool` (True es un entero en Python) y valores no finitos como `inf`, además de los negativos.
- **Copia al listar.** `listar_productos()` devuelve `.copy()`; así nadie vacía el inventario desde fuera. Los objetos siguen siendo los mismos (copia superficial), por eso Producto protege sus datos.

- **Reorganización de responsabilidades.** Inventario devuelve objetos y booleanos; los textos «Producto registrado.», «ya existe», etc. se deciden únicamente en `main.py`.
- **Precios con puntos.** En el Laboratorio 2 el precio escrito como 85.000 se leyó como 85.0 y el resumen salió equivocado. En esta versión `convertir_precio` interpreta 85.000 como 85000 y 85.000,50 como 85000.5, y hay una prueba unitaria que lo comprueba.
- **Categorías en orden estable.** Un conjunto no garantiza orden; `main.py` usa `sorted()` solo para mostrarlas, sin modificar el conjunto que devuelve Inventario.

Transparencia sobre el historial. El historial fue armado a partir de los archivos del laboratorio ya terminados, con ayuda de Claude (los commits incluyen la línea Co-Authored-By). Por eso las fechas de los commits son las de su creación y no las de cada práctica.

5. Comparación técnica: gestor procedural (Lab 2) vs. orientado a objetos (Lab 3)

Aspecto	Laboratorio 2 (procedural)	Laboratorio 3 (POO)
Organización	Un archivo (gestor_inventario.py, 203 líneas) con un ciclo while y todo el código en él.	Tres módulos: producto.py (81), inventario.py (74), main.py (196).
Representación del producto	Diccionario {"codigo": ..., "precio": ...}; cualquier parte puede cambiar o dejar claves.	Objeto Producto con atributos internos y propiedades.
Validación	Condiciones dentro del menú, al momento de registrar. Otras partes del programa podrían guardar valores inválidos.	En Producto.__init__ y en actualizar_existencias: se aplican siempre, sin importar quién cree el objeto.
Protección de datos	Ninguna: producto["cantidad"] = -5 es posible.	_cantidad solo cambia mediante un método que valida; el código no tiene setter.
Reutilización	La lógica depende de input() y print(); no se puede usar sin el menú.	Producto e Inventario no tienen entrada/salida; se pueden probar y reutilizar solos.
Pruebas	Solo ejecutando el menú a mano.	31 pruebas automáticas sobre clases y menú.
Varias colecciones	Una sola lista global productos.	Cada Inventario() tiene su lista; se pueden tener varios.
Cálculo del valor	Acumulador dentro del ciclo del resumen.	Producto.calcular_valor() devuelve el dato; Inventario lo suma.
Mantenimiento	Un cambio de regla obliga a revisar varias partes del bloque if/elif.	Un cambio de regla se hace en un solo método de la clase responsable.

Las funcionalidades son las mismas, pero en la versión POO cada regla vive en el lugar al que pertenece (el producto sabe validarse, el inventario sabe administrar productos y main sabe hablar con el usuario). El costo es más archivos y algo más de código; el beneficio es que los errores se localizan mejor y los datos están protegidos.

6. Respuestas a las preguntas de análisis situacional

Pregunta 1

¿Qué harías si cualquier parte del programa puede modificar directamente el precio o la cantidad de un producto, incluso asignando valores negativos?

Qué haría. Encapsularía los atributos: los guardaría como internos (`_precio`, `_cantidad`), los expondría solo por propiedades de lectura y permitiría cambiarlos únicamente mediante métodos o setters que validen.

Por qué. Con atributos públicos, `teclado.cantidad = -10` es válido para Python, pero no para el negocio. Si la regla vive en un solo punto de entrada, no depende de que cada parte del programa recuerde aplicarla.

Cómo lo implementaría. Atributos con guion bajo; `@property` sin setter para código, cantidad y precio; `actualizar_existencias()` que devuelve False si el valor es negativo o no es entero; y las mismas validaciones en `__init__` para que el objeto no nazca inválido (el proyecto usa esta variante). Si se necesitara cambiar el precio, un setter con `raise ValueError` para valores ≤ 0, como en la práctica 3.4.

```
@property
def cantidad(self):
    return self._cantidad

def actualizar_existencias(self, nueva_cantidad):
    if isinstance(nueva_cantidad, bool) or not isinstance(nueva_cantidad, int) or nueva_cantidad < 0:
        return False
    self._cantidad = nueva_cantidad
    return True
```

Cómo lo comprobaría. Pruebas `test_actualizacion_de_cantidad` (10 se acepta, -3 se rechaza y la cantidad sigue en 10), `test_rechaza_datos_iniciales_invalidos` (siete combinaciones inválidas lanzan ValueError) y `test_cantidad_cero_y_codigo_solo_lectura` (asignar codigo produce AttributeError). Todas aprobadas.

Pregunta 2

¿Qué harías si al crear dos objetos Inventario y agregar un producto al primero, el producto también aparece en el segundo?

Qué haría. Revisaría dónde se crea la lista de productos. El síntoma indica que está declarada en el cuerpo de la clase (atributo de clase) en lugar de dentro de `__init__` (atributo de instancia).

Por qué. Una lista mutable en el cuerpo de la clase es un único objeto compartido por todas las instancias, así que append sobre una se ve desde la otra.

Cómo lo implementaría. Eliminar `_productos = []` del nivel de clase y crearla con `self._productos = []` dentro de `__init__`. Además, listar devuelve una copia para no exponer la lista interna.

```
class Inventario:
    def __init__(self):
        self._productos = []   # una lista nueva por cada objeto
```

Cómo lo comprobaría. `pruebas_independencia.py` imprime «Primero: 1» y «Segundo: 0»; la prueba `test_inventarios_independientes` lo verifica con assert. En `gestor_depuracion.py` (error 1) reproduje el fallo con la lista a nivel de clase: el segundo inventario mostraba ['Teclado']; con la corrección, [].

Pregunta 3

¿Qué harías si el programa necesita mostrar información de un producto, pero la clase Producto contiene input() y controla opciones del menú?

Qué haría. Sacaría de Producto toda la interacción y repartiría las responsabilidades: Producto solo guarda y valida datos y ofrece métodos que devuelven valores; Inventario administra la colección;

`main.py` pide datos, muestra el menú y los mensajes.

Por qué. Si una clase de dominio lee del teclado o imprime, no se puede reutilizar ni probar sin simular a un usuario, y mezcla reglas de negocio con presentación. Separarlas hace que cada cambio afecte a un solo lugar.

Cómo lo implementaría. Producto recibe los datos por parámetros en el constructor y expone propiedades y `__str__`; main.py hace `input()`, construye el Producto dentro de un `try/except ValueError`, se lo pasa a Inventario y según el resultado (True/False/None) decide qué imprimir.

```
# main.py (interacción)
try:
    producto = Producto(codigo, nombre, categoria, precio, cantidad)
except ValueError as error:
    print("No se pudo registrar el producto:", error)
    return
print("Producto registrado." if inventario.registrar_producto(producto)
      else "No se pudo registrar: el código ya existe.")
```

Cómo lo comprobaría. Busqué en producto.py e inventario.py: no contienen input() ni menús. Las pruebas unitarias crean Producto e Inventario sin simular teclado, y `pruebas_obligatorias.py` solo necesita simular input() para probar main.py.

Pregunta 4

¿Qué harías si un método debe calcular el valor económico de un producto, pero solo imprime el resultado y no permite usarlo en el resumen general?

Qué haría. Cambiaría `print()` por `return`: el método devuelve el número y quien lo llama decide si lo imprime, lo suma o lo compara.

Por qué. Un valor impreso solo se ve en pantalla; un valor devuelto puede reutilizarse (práctica 3.1: `mostrar_total` vs. `obtener_total`).

Cómo lo implementaría. El método necesita solo el estado del objeto (`self._precio` y `self._cantidad`); no requiere parámetros. Inventario aprovecha el valor devuelto sumando el de cada producto con una expresión generadora.

```
# producto.py
def calcular_valor(self):
    return self._precio * self._cantidad

# inventario.py (dentro de resumen)
"valor_total": sum(p.calcular_valor() for p in self._productos),
```

Cómo lo comprobaría. Con los tres productos del enunciado el resumen da $1740000.0; al actualizar P002 a 5 unidades da $1965000.0, igual al cálculo manual. En el reto de depuración (error 6) sumar solo precios daba 785000.0; con calcular_valor() da 1740000.0.

Pregunta 5

¿Qué harías si, después de separar el proyecto en varios módulos, aparece un error de importación y el programa principal deja de ejecutarse?

Qué haría. Leería el mensaje del error completo (ModuleNotFoundError, ImportError o NameError) y revisaría tres cosas: la estructura de archivos, las instrucciones `import` y el historial de GitHub.

Por qué. Estos errores suelen venir de un archivo movido o renombrado, un nombre mal escrito, una ejecución desde otra carpeta, un import eliminado o un archivo propio que se llama igual que un módulo estándar (json.py, math.py).

Cómo lo implementaría. Estructura: confirmar que main.py, producto.py e inventario.py están en la misma carpeta y ejecutar desde `gestor_inventario/`. Imports: comprobar que son `from producto import Producto` y `from inventario import Inventario`, con los nombres escritos igual que los archivos y las clases, y que no hay importaciones circulares. GitHub: usar `git log`, `git diff` o `git status` para ver qué cambió en el último commit que funcionaba y restaurar con `git restore <archivo>` si hace falta.

```
cd laboratorio-03/gestor_inventario
python main.py
git log --oneline -- inventario.py        # qué commits tocaron el archivo
git diff HEAD~1 -- main.py                # qué se cambió en los imports
```

Cómo lo comprobaría. Ejecutaría de nuevo `python main.py` y las pruebas. En `gestor_depuracion.py` (error 5) reproduje el problema: sin el import aparece «name 'Inventario' is not defined»; con la línea restaurada, el objeto se crea.

7. Conclusión

El Laboratorio 3 transformó el gestor procedural en una aplicación con clases y módulos. Se conservaron todas las funcionalidades, se añadieron validaciones desde la creación de los objetos y la interacción quedó aislada en `main.py`. Las 31 pruebas automáticas pasan y los resultados numéricos coinciden con los del laboratorio. Repositorio: https://github.com/skyyleerr/python-laboratorios.